Lab02

Max Stróżyk

Winter 2024/2025

English translation of the Polish original. Historical results and formulas retained.

Contents

1 Task 1 **2**

2 Task 2 **3**

3 Task 3 **4**

4 Task 4 **5**

5 Task 5 **6**

1 Task 1

We compare different algorithms for matrix multiplication. **Operation of mm**:

1. loop over the row indices of matrix A
2. loop over the column indices of matrix B
3. loop over the column indices of matrix A
4. accumulate each element by multiplying the corresponding row and column entries

Operation of mm1:

1. loop over the row indices of matrix A
2. loop over the column indices of matrix B
3. multiply the corresponding rows of A and columns of B

Default matrix multiplication:
I could not determine how it is implemented; most likely, its loops are organized differently.

n	mm	mm1	default
100	5.28124s	0.052098s	0.000175953
500	671.192s	1.42004s	0.00494003

Conclusions:
Adding another loop significantly increases computational complexity. In the first case the complexity is n^3, and in the second it is n^2. The default algorithm must use a different approach: it is substantially faster than our implementation.

2 Task 2

We compare algorithms for solving lower-triangular systems. **Operation of SolveL**

1. loop over index i from 2 to the length of x
2. loop over indices from 1 to i
3. accumulate products of successive elements

Operation of SolveL1

1. loop over row indices
2. accumulate using matrix multiplication

n	SolveL	error	SolveL1	error
1000	1.43726s	$6.2607e+159$	0.00959992s	$1.2082e+159$
10000	135.994s	$1.9449e+295$	0.278352s	$1.8671e+295$
1000 + eye	1.36324s	$4.7684e-06$	0.00939488s	$6.4969e-06$

Solving lower-triangular systems is very fast. Attention must be paid to the measured error. It comes from a product of many small numbers, which tends to 0. When dividing by the reciprocal, we obtain very large errors.

3 Task 3

We test an algorithm that computes the inverse of an upper-triangular matrix.
Operation of invU

- loop over columns
- loop over rows
- accumulate products of successive elements

n	invU
100	0.906316s
1000	1.43726s

Inverting upper-triangular matrices is very fast.

4 Task 4

We investigate the error of the LU decomposition of a matrix.

matrix	time	error
A	0.000115156	0
B	0.000115871	0
C	0.000169039	0
random 100x100	2.04939	1.5188e-13

The errors in computing the matrices are negligible. I was unable to reproduce the errors observed in class.

5 Task 5

LLT decomposition

n	time	error
10	0.000947952s	$4.4409e-16$
100	0.0534279	$3.9345e+24$
1000	8.54853	$6.3200e+296$

The computation is unstable for large matrices, despite increasing the main-diagonal entries. Runtime performance is satisfactory. Why are these errors so large?